Proof-of-Concept Report: Adaptive Behavioral History Management for Resource-Efficient Short-Video Recommendation

Nguyễn Quang Trung Hiếu

October 14, 2026

Abstract

Behavioral history is an important source of information for short-video recommendation. However, longer histories require more interactions to be stored, retrieved, and processed, while additional interactions do not necessarily provide proportional improvements in predictive performance. This proof of concept (PoC) investigates whether limiting the amount of recent behavioral history can reduce data-processing requirements while retaining an acceptable level of recommendation-related predictive performance. The implementation uses existing data artifacts and baseline evaluation results from the KuaiRand-1K dataset. It introduces a history-budget selection policy based on validation ROC-AUC and benchmarks history processing for the selected configuration against a full-history reference. The history budget is selected from $K \in \{0, 1, 5, 10, 20, 50, 100, \text{Full}\}$. With an allowable validation AUC decrease of $\epsilon = 0.05$, the policy selects $K = 100$, the smallest tested budget satisfying the predefined validation constraint. On 43,027 evaluation requests, this configuration contains 4,079,877 history interactions, compared with 24,721,035 for Full, representing an 83.50% reduction. A repeated local streaming benchmark records median processing times of 0.512 seconds for $K = 100$ and 2.895 seconds for Full, a measured reduction of 82.31%. The existing baseline results show a test ROC-AUC of 0.579012 for $K = 100$, compared with 0.614246 for Full. Thus, the observed reduction in history volume comes with a measurable decrease in predictive discrimination. These results demonstrate the initial feasibility of a global history-budget selection policy. They do not establish serverless cost savings, energy savings, or per-user adaptive recommendation.

Keywords: KuaiRand, short-video recommendation, behavioral history, history budget, ROC-AUC, mean pooling, resource efficiency, proof of concept.

1 Introduction

1.1 Background and motivation

Short-video recommendation systems use users' previous interactions to estimate their interests and predict how they may respond to candidate videos. A behavioral history may contain hundreds or thousands of interactions, but not every interaction necessarily contributes equally to the prediction.

Recent interactions may provide useful signals about a user's current interests, whereas older interactions may contain redundant or less relevant information. Consequently, providing a model with more history does not guarantee a proportional increase in predictive performance.

At the same time, processing longer histories can require more data access and computation. This motivates a practical question:

> *Can a recommendation model use a smaller amount of recent behavioral history while retaining an acceptable level of predictive performance?*

The original research direction, *How Much Intelligence Can We Get for How Little?*, approaches this question by treating history length as a controlled experimental variable. The PoC extends this direction with a preliminary resource-efficiency investigation.

1.2 Objectives

The PoC has three objectives:

1. **History-budget selection:** Select the smallest tested history budget that satisfies a predefined validation AUC constraint.
2. **History-volume measurement:** Quantify how limiting history changes the total number of interactions represented in the evaluation histories.
3. **Runtime benchmarking:** Measure the time required to read and process the stored history data under a controlled local benchmark.

The project reuses existing baseline models, checkpoints, and evaluation results. The benchmark stage does not retrain the models.

1.3 Scope

This work is a proof of concept rather than a production recommendation system. It focuses on a global history-budget policy and the processing of precomputed history artifacts.

It does not introduce a new state-of-the-art recommendation architecture, perform causal treatment-effect estimation, or deploy a complete adaptive recommendation pipeline on a serverless platform.

2 Dataset and Evaluation Task

2.1 Dataset

The PoC uses KuaiRand-1K, a dataset derived from short-video recommendation logs from Kuaishou. It contains chronological user–video interactions and multiple observable feedback signals, including clicks, likes, follows, comments, play time, and long-view outcomes.

The processed evaluation artifacts contain:

- Evaluation events: 43,027
- Unique users: 1,000
- Unique observed videos: 7,388
- Positive long-view labels: 3,621
- Negative long-view labels: 39,406
- Positive-label rate: 8.42%

All 43,027 evaluation events in the inspected artifact have `is_rand = 1`, indicating that the evaluation set consists of random-intervention observations.

These observations are useful for studying predictive discrimination under observed exposures. However, random intervention should not be interpreted as eliminating every source of exposure bias or providing outcomes for every possible candidate video.

2.2 Prediction target

The target variable is `long_view`, a binary label supplied by KuaiRand:

$$y \in \{0, 1\} \tag{1}$$

The model predicts the probability of a long-view outcome for an observed user–video exposure.

The primary evaluation task is therefore to distinguish observed positive long-view outcomes from observed negative outcomes. It is not a complete evaluation of a recommendation policy over every video that could have been recommended.

2.3 Why use long-view?

The project uses the dataset's existing `long_view` label rather than defining a new engagement threshold.

This choice avoids introducing an additional subjective threshold for watch duration or watch ratio. It also keeps the target definition consistent across history-budget configurations.

3 Behavioral History Construction

3.1 Temporal eligibility

For an evaluation event at time t, let H_t denote the eligible behavioral history. Only interactions occurring strictly before the evaluation timestamp are allowed:

$$H_t = \{e_i < t\} \tag{2}$$

The implemented history window is one day. More precisely, eligible events satisfy:

$$t - 24 \text{ hours} \leq t_i < t \tag{3}$$

This constraint prevents future interactions from being included in the history used to predict an earlier outcome.

The one-day window also limits the historical context to a consistent temporal range. It was used as a fixed configuration in this PoC rather than being optimized during the benchmark.

3.2 History budgets

The finite history budgets are:

$$K \in \{0, 1, 5, 10, 20, 50, 100\} \tag{4}$$

For a positive finite budget, the history contains at most the K most recent eligible interactions:

$$H_t^{(K)} = \text{LastK}(H_t) \tag{5}$$

For $K = 0$, no behavioral history is supplied.

The Full configuration contains the available history within the one-day window, truncated to at most 2,048 interactions:

$$\text{Last}_{2048}(H_t) \tag{6}$$

Thus, Full is a bounded full-history reference within the predefined window, not unlimited access to every interaction in the dataset.

3.3 History reconstruction and reuse

The history-generation pipeline reconstructs the one-day history once and derives the finite-budget configurations by slicing that history.

This approach is useful for two reasons:

- It avoids repeating the full history-reconstruction process for every value of K.
- It ensures that configurations are derived from the same underlying historical context.

The same evaluation event set is used across the history-budget configurations, so the comparison is not intentionally driven by changing the evaluation population.

3.4 Padding and masking

The model implementation represents histories as tensors. Because histories have different lengths, shorter sequences are padded to a common length within a dataset split.

Padding uses index zero, while valid interactions are marked by a binary history mask:

$$m_{ij} = \begin{cases} 1 & \text{if interaction } j \text{ is valid} \\ 0 & \text{if interaction } j \text{ is padding} \end{cases} \tag{7}$$

The mask prevents padding entries from contributing to the aggregated history representation.

A frozen video vocabulary is used to map raw video identifiers to model indices, with zero reserved for padding. This keeps identifier encoding consistent across the compared configurations.

4 Baseline Model Architecture

4.1 Model overview

The baseline model is a lightweight embedding-based neural recommender with masked mean pooling and a multilayer perceptron (MLP) classifier. It is implemented in PyTorch as `KuaiRandRecommender`.

The main components are:

1. Candidate video embedding.
2. Historical interaction embeddings.
3. Masked mean-pooling aggregation.
4. Feature concatenation.
5. MLP classification head.
6. Sigmoid probability output.

The model is intentionally simple. The main research variable is the amount of behavioral history, not architectural complexity. Using a lightweight baseline makes it easier to compare history budgets while keeping the modeling approach fixed.

4.2 Candidate video embedding

Each candidate video is mapped to a trainable 64-dimensional embedding:

$$\mathbf{v} \in \mathbb{R}^{64} \tag{8}$$

The embedding represents the candidate video using its identifier. The inspected baseline architecture does not combine additional video metadata such as duration or author features in this embedding module.

4.3 Historical interaction embeddings

Each historical video identifier is mapped to a 64-dimensional embedding:

$$\mathbf{e}_j \in \mathbb{R}^{64} \tag{9}$$

For a history of length L, the model obtains a sequence of historical embeddings. The history encoder and candidate encoder are separate embedding modules in the implementation.

This design provides a straightforward representation of historical interactions without introducing a recurrent network or Transformer.

4.4 Masked mean pooling

The historical embeddings are aggregated into a single user-state vector using masked mean pooling:

$$\mathbf{h}_t^{(K)} = \frac{\sum_{j=1}^{L} m_j \mathbf{e}_j}{\max\left(1, \sum_{j=1}^{L} m_j\right)} \tag{10}$$

where:

- L is the padded sequence length;
- $\mathbf{e}_j$ is the embedding of the j-th historical interaction;
- m_j is the validity mask.

The result is a 64-dimensional history representation.

Mean pooling was chosen because it is simple, computationally lightweight, and keeps the output dimension independent of history length. As a result, changing K does not require changing the downstream classifier's input dimension.

An important limitation is that mean pooling does not explicitly model interaction order or temporal dependencies between individual events. This is acceptable for the current PoC because the primary goal is to investigate the effect of history volume, not to compare sequence-model architectures.

For an empty history, the mask produces a zero history representation.

4.5 Feature fusion and classification

The history representation and candidate embedding are concatenated:

$$[\mathbf{h}_t^{(K)}; \mathbf{v}] \in \mathbb{R}^{128} \tag{11}$$

The MLP classifier has two hidden layers:

$$128 \to 128 \to 64 \to 1 \tag{12}$$

Each hidden layer uses ReLU activation and batch normalization. The final layer produces a scalar logit, followed by a sigmoid activation:

$$\hat{y} = \sigma(g_\theta(\mathbf{z})) \tag{13}$$

where $\hat{y}$ is the predicted probability of a long-view outcome.

For the main history-budget comparison, the intended invariant is that the model architecture and training protocol remain fixed while only the available history changes. The $K = 0$ condition has no valid history contribution.

4.6 Why not use a Transformer or a more complex model?

A more complex sequential model could potentially capture temporal order and interactions between historical events. However, introducing such a model would also introduce additional architectural choices, parameters, and computational requirements.

The lightweight baseline is more appropriate for this initial PoC because it:

- isolates history budget as the main variable of interest;
- avoids introducing unnecessary architectural complexity;
- makes the experiment easier to reproduce;
- keeps model development and evaluation computationally manageable.

The results should therefore be interpreted as findings for this particular baseline and protocol, not as universal conclusions about every recommendation model.

5 Training Procedure

5.1 Training configuration

The inspected baseline training implementation uses the following settings:

Component	**Configuration**
Framework	PyTorch
Embedding dimension	64
MLP hidden dimensions	128, 64
Hidden activation	ReLU
Normalization	BatchNorm1d
Output activation	Sigmoid
Loss function	Binary cross-entropy
Optimizer	Adam
Learning rate	0.001
Batch size	2,048
Maximum epochs	20
Early-stopping patience	3 epochs
Model-selection metric	Validation ROC-AUC

>

The pre-existing training artifacts use train, validation, and test splits containing 30,118, 6,454, and 6,455 evaluation records, respectively. The total is 43,027 records.

The split-generation procedure should be documented separately before claiming that these are chronological partitions. This PoC report does not assume a temporal split solely from the split sizes.

5.2 Why binary cross-entropy?

The target `long_view` is binary, and the model outputs a probability $\hat{y}$ through a sigmoid activation. Binary cross-entropy is therefore used as the training objective:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right] \tag{14}$$

This loss penalizes incorrect probability estimates for positive and negative outcomes and provides a direct objective for binary classification.

Binary cross-entropy is the optimization loss; it is not the primary metric used to choose the history budget.

5.3 Why Adam?

Adam is used as the optimizer with a learning rate of 10^{-3}. It is a practical default for training embedding-based neural networks and MLPs, combining adaptive per-parameter learning rates with momentum-like estimates.

The PoC does not claim that Adam is optimal. The intent is to use a fixed, workable optimization configuration rather than expand the experiment into an optimizer comparison.

5.4 Early stopping and checkpoint selection

Training runs for up to 20 epochs, with early-stopping patience set to three epochs.

The checkpoint with the best validation ROC-AUC is retained. This limits unnecessary training after validation performance stops improving and provides a consistent selection rule for the existing baseline models.

For the current PoC, these training runs and checkpoints are already available. The new history-processing benchmark does not retrain them.

6 Evaluation Metrics and Selection Policy

6.1 Primary predictive metric: ROC-AUC

The primary evaluation metric is the area under the receiver operating characteristic curve (ROC-AUC).

ROC-AUC measures how well model scores discriminate between positive and negative examples across classification thresholds. It is suitable for this task because the model produces engagement scores, and the experiment is concerned with ranking quality rather than selecting one fixed classification threshold.

A ROC-AUC of 0.5 corresponds to random discrimination under the usual interpretation, while higher values indicate stronger discrimination.

The dataset has an observed positive-label rate of approximately 8.42%. ROC-AUC provides a threshold-independent discrimination metric, but it does not fully describe performance on an imbalanced dataset. PR-AUC could provide complementary information in a future evaluation; it is not reported as a result of this PoC.

Importantly, the measured ROC-AUC describes predictive discrimination on the observed evaluation exposures. It is not equivalent to causal policy value, complete counterfactual ranking quality, or production recommendation success.

6.2 Why use validation AUC to choose K?

The history budget is selected by comparing validation ROC-AUC against the Full reference.

Let:

- $A_{\text{val}}(K)$ be the validation ROC-AUC for budget K;
- $A_{\text{val}}(\text{Full})$ be the validation ROC-AUC for Full;
- ϵ be the maximum allowed decrease.

The selection rule is:

$$K^* = \min \{K : A_{\text{val}}(K) \geq A_{\text{val}}(\text{Full}) - \epsilon\} \tag{15}$$

The PoC uses:

$$\epsilon = 0.05 \tag{16}$$

Since the Full validation AUC is 0.659740, the minimum acceptable validation AUC is:

$$0.659740 - 0.05 = 0.609740 \tag{17}$$

This rule is a simple, interpretable constraint-based selection method. It identifies the smallest tested budget that satisfies a predefined quality tolerance.

The value $\epsilon = 0.05$ is a design choice for this PoC, not a universally established acceptable loss in recommendation quality.

6.3 Why use validation rather than test AUC for selection?

Using test AUC to choose the history budget would allow the final evaluation data to influence the policy decision.

The implementation therefore selects K using validation AUC and reports test AUC afterward as an evaluation result. This separates the budget-selection rule from the final reported test performance.

6.4 Engineering metrics

In addition to ROC-AUC, the PoC reports three measurements:

- **Total history interactions:** The total number of stored history entries across the 43,027 evaluation requests. This directly measures the history volume represented by each configuration.
- **Mean interactions per request:**

$$\frac{\text{Total history interactions}}{\text{Number of requests}} \tag{18}$$

  This provides an interpretable estimate of the average history length.

- **Local history-processing time:** The benchmark reads the Parquet history column in batches of 256, converts each batch to Python lists, and counts the contained interactions. The elapsed time includes reading and processing the selected history column. This measurement is used as a local proxy for the cost of processing history data. It is not a measurement of complete model inference latency or serverless execution cost.

7 Experimental Results

7.1 History-budget selection

The existing validation results are:

The $K = 50$ configuration does not reach the required threshold of 0.609740. The $K = 100$ configuration reaches 0.611039 and therefore satisfies the selection rule.

Consequently, $K = 100$ is the smallest tested history budget satisfying the validation constraint.

K	Validation ROC-AUC	Meets threshold?
0	0.590352	No
1	0.507195	No
5	0.543251	No
10	0.558826	No
20	0.578821	No
50	0.598103	No
100	0.611039	Yes
Full	0.659740	Yes

Metric	K = 100	Full
Validation ROC-AUC	0.611039	0.659740
Test ROC-AUC	0.579012	0.614246
Test AUC gap relative to Full	0.035234	0

7.2 Predictive performance

The validation AUC gap for $K = 100$ is:

$$0.659740 - 0.611039 = 0.048701 \tag{19}$$

This is smaller than the allowed decrease of 0.05, which explains why the configuration is selected.

On the test set, the AUC gap is:

$$0.614246 - 0.579012 = 0.035234 \tag{20}$$

Thus, the selected configuration retains useful predictive discrimination but performs worse than Full on the reported test AUC. The policy satisfies the chosen validation constraint; it does not establish predictive equivalence between $K = 100$ and Full.

7.3 History-volume comparison

The history-processing artifacts contain the following totals:

Metric	K = 100	Full
Evaluation requests	43,027	43,027
Total history interactions	4,079,877	24,721,035
Mean interactions per request	94.82	574.55

The reduction is calculated as:

$$\left(1 - \frac{N_{100}}{N_{\text{Full}}}\right) \times 100\% \tag{21}$$

Substituting the observed totals:

$$\left(1 - \frac{4,079,877}{24,721,035}\right) \times 100\% \approx 83.50\% \tag{22}$$

Therefore, the $K = 100$ artifacts contain approximately 83.50% fewer history interactions than the Full artifacts for the same set of evaluation requests.

This is a direct measurement of history volume. It should not be interpreted as an equivalent reduction in RAM usage, energy consumption, or total system cost.

7.4 Repeated runtime benchmark

The benchmark was run four times for each configuration, alternating execution order between $K = 100$ and Full.

Runtime statistic	K = 100	Full
Number of runs	4	4
Minimum time	0.498 s	2.864 s
Median time	0.512 s	2.895 s
Maximum time	0.530 s	3.029 s
Time range	0.032 s	0.165 s

The median runtime reduction is:

$$\left(1 - \frac{T_{100}}{T_{\text{Full}}}\right) \times 100\% = \left(1 - \frac{0.512}{2.895}\right) \times 100\% \approx 82.31\% \tag{23}$$

The median Full processing time is approximately 5.65 times the median $K = 100$ processing time.

The four measurements show relatively small ranges for both configurations. Nevertheless, operating-system caching, execution order, and background system activity may affect the results. Four runs are sufficient for an initial PoC comparison, but they do not establish a universal runtime improvement.

7.5 Consolidated comparison

Dimension	K = 100	Full
Validation ROC-AUC	0.611039	0.659740
Test ROC-AUC	0.579012	0.614246
History interactions	4,079,877	24,721,035
Median local processing time	0.512 s	2.895 s

The results reveal a trade-off. Limiting history substantially reduces the number of interactions that must be processed and reduces measured local history-processing time, but it also decreases test predictive performance.

8 Discussion

8.1 What the PoC demonstrates

The PoC demonstrates that a validation-AUC-based rule can select a smaller history budget from a predefined set of configurations.

For the current baseline, the selected budget is $K = 100$. Compared with Full, it reduces the stored history interactions by 83.50%, while satisfying the validation AUC tolerance specified in advance.

The repeated local benchmark also shows that processing the smaller history artifacts takes less time under the tested conditions.

Together, these results provide initial evidence that history-budget selection is a practical direction for investigating the relationship between predictive quality and history-processing requirements.

8.2 What the results do not demonstrate

The measured runtime reduction must not be interpreted as an 82.31% reduction in serverless latency or cost.

The benchmark measures reading, decoding, and iterating through the stored history column on a local machine. It does not include a complete recommendation request, model inference, network access, function initialization, or serverless resource billing.

Similarly, the 83.50% reduction in history interactions does not imply an identical reduction in memory usage or energy consumption.

8.3 Why the current policy is global rather than fully adaptive

The current policy chooses one budget from the baseline validation results and applies that budget globally. It does not compute a different K for each user or request.

Therefore, the implementation is best described as a global history-budget selection policy. Although it supports the broader motivation of adaptive behavioral history management, it is not yet a per-user or per-request adaptive system.

This distinction is important when interpreting the scope and contribution of the current PoC.

9 Limitations

The current implementation has the following limitations:

1. **Simple baseline architecture:** The model uses video embeddings, masked mean pooling, and an MLP. It does not explicitly model sequential order or long-range temporal dependencies.

2. **Single history-window configuration:** The benchmark uses a one-day history window and a maximum Full length of 2,048 interactions. Other windows and truncation settings have not been compared in this PoC.

3. **Existing baseline results:** The reported AUC values are reused from existing training artifacts. The runtime benchmark does not retrain models or independently reproduce their predictive metrics.

4. **Limited runtime measurements:** Each configuration has four local benchmark runs. The results may be affected by caching and local machine conditions.

5. **No serverless deployment:** No actual serverless function execution, cold-start or warm-start experiment, autoscaling test, or billing-based cost measurement has been conducted.

6. **No energy measurement:** The PoC does not measure electrical energy or establish energy savings.

7. **No per-request adaptation:** The selected budget is global rather than dynamically determined from each user's or request's characteristics.

8. **Evaluation scope:** ROC-AUC measures discrimination on observed intervention exposures. It does not establish causal effects or complete counterfactual recommendation-policy quality.

9. **Split-generation documentation:** The inspected Phase 8 artifacts provide the sizes of the train, validation, and test splits. The exact split-generation procedure should be verified before describing the split as chronological.

10. **Baseline dependence:** The selected budget depends on the current model, dataset, evaluation protocol, and chosen tolerance $\epsilon = 0.05$. It should not be assumed to be optimal for other models or datasets.

These limitations define the boundary between the implemented PoC and a more complete research system.

10 Conclusion

This proof of concept investigates the feasibility of reducing behavioral-history processing requirements in short-video recommendation while maintaining a predefined level of predictive performance.

Using existing KuaiRand-1K artifacts and baseline evaluation results, the implemented selection rule identifies $K = 100$ as the smallest tested history budget satisfying the validation ROC-AUC constraint with $\epsilon = 0.05$.

Across 43,027 evaluation requests, the selected configuration contains 4,079,877 history interactions, compared with 24,721,035 for Full, corresponding to an 83.50% reduction. A repeated local streaming benchmark records median processing times of 0.512 seconds and 2.895 seconds, respectively, corresponding to an 82.31% reduction in the measured operation.

The test ROC-AUC for $K = 100$ is 0.579012, compared with 0.614246 for Full. The results therefore demonstrate a trade-off rather than an unconditional improvement: the smaller history reduces the amount of data processed and the measured local processing time, while predictive discrimination decreases.

Overall, the PoC provides a reproducible starting point for studying history-budget selection. Its results support further investigation but do not yet establish serverless cost savings, energy savings, or per-request adaptive behavior.

A Key Implementation Artifacts

The following repository artifacts are relevant to reproducing or extending the PoC:

- `src/data/build_history_configs.py` — reconstructs the one-day history and derives history-budget configurations.
- `data/processed/phase7/history_config.yaml` — records the history window, maximum history length, and tested budgets.
- `src/models/baseline_models.py` — defines the baseline embedding, mean-pooling, and MLP architecture.
- `src/models/train_baseline.py` — defines the baseline training procedure.
- `data/processed/phase8/baseline_results.csv` — stores the existing baseline AUC results.
- `src/policy/adaptive_policy.py` — implements the validation-AUC-based global history-budget selection policy.
- `src/policy/benchmark_history.py` — measures one streaming pass over the selected history column.
- `src/policy/benchmark_history_repeated.py` — runs the repeated local benchmark.
- `src/policy/build_tradeoff_table.py` — combines predictive, history-volume, and runtime results.

- `data/processed/phase8/adaptive_policy_summary.csv` — stores the policy summary.
- `data/processed/phase8/runtime_benchmark.csv` — stores the initial single-pass benchmark results.
- `data/processed/phase8/runtime_benchmark_repeated.csv` — stores the individual repeated benchmark runs.
- `data/processed/phase8/runtime_benchmark_summary.csv` — stores the repeated benchmark summary.
- `data/processed/phase8/tradeoff_summary.csv` — stores the consolidated trade-off table.

B Main Equations

Validation-constrained history-budget selection:

$$K^* = \min\{K : A_{\text{val}}(K) \geq A_{\text{val}}(\text{Full}) - \epsilon\} \tag{24}$$

Masked mean pooling:

$$\frac{\sum_j m_j \mathbf{e}_j}{\max(1, \sum_j m_j)} \tag{25}$$

History-interaction reduction:

$$\left(1 - \frac{N_{100}}{N_{\text{Full}}}\right) \times 100\% \tag{26}$$

Local processing-time reduction:

$$\left(1 - \frac{T_{100}}{T_{\text{Full}}}\right) \times 100\% \tag{27}$$